FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Adam Balko

Arduino platform simulator

Department of Software and Computer Science Education

Supervisor of the bachelor thesis: RNDr. David Bednárek, Ph.D.

Study programme: Computer Graphics, Vision and Computer Games Development

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Arduino platform simulator

Author: Adam Balko

Department: Department of Software and Computer Science Education

Supervisor: RNDr. David Bednárek, Ph.D., Department of Software Engineering

Abstract: Use the most precise, shortest sentences that state what problem the thesis addresses, how it is approached, pinpoint the exact result achieved, and describe the applications and significance of the results. Highlight anything novel that was discovered or improved by the thesis. Maximum length is 200 words, but try to fit into 120. Abstracts are often used for deciding if a reviewer will be suitable for the thesis; a well-written abstract thus increases the probability of getting a reviewer who will like the thesis.

Keywords: digital circuit simulator, Arduino

Název práce: Simulátor prostředí Arduino

Autor: Adam Balko

Katedra: Katedra softwaru a výuky informatiky

Vedoucí bakalářské práce: RNDr. David Bednárek, Ph.D., Katedra softwarového inženýrství

Abstrakt: Abstrakt práce přeložte také do češtiny.

Klíčová slova: simulátor číslicových obvodů, Arduino

Contents

Introduction **6**
0.1 Comparison of existing solutions . . . 6
0.2 Specific goals of the thesis . . . 6

1 Architecture design **7**
1.1 Architecture overview . . . 7
1.2 Inter-process communication . . . 7
1.2.1 Events . . . 8
1.2.2 TCP stream . . . 8
1.2.3 Memory mapping . . . 8
1.3 Backend . . . 8
1.4 Frontend . . . 8

2 Implementation **9**
2.1 Backend . . . 9
2.1.1 Simulator module . . . 9
2.1.2 Executable module . . . 9
2.1.3 IPC Adapters . . . 9
2.2 Frontend . . . 9
2.2.1 Component Management project . . . 9
2.2.2 Main project with Avalonia UI . . . 9
2.2.3 IPC Adapter project . . . 9

3 User documentation **10**
3.1 Compiling and running Arduino code . . . 10
3.2 Defining components . . . 10
3.3 Defining scenes . . . 10

Conclusion **11**

List of Figures **12**

List of Tables **13**

List of Abbreviations **14**

A Attachments **15**
A.1 First Attachment . . . 15

Introduction

Arduino is a very popular microcomputer among amateurs and professionals alike. Despite this, testing the code for an Arduino machine can be fairly difficult. Debugging options are limited and the user needs to rely on additional measuring equipment. The usual choice is to test the code with an AVR emulator. Emulation is the process of running machine code on a hardware that doesn't match its instruction set. AVR is the processor that controls the Arduino board. These emulators are sometimes part of a bigger logical circuit simulation system, other times they are just a single piece of software.

Goal of this work is to skip the emulation process and create a working Arduino simulator by mimicking the changes of digital values on the board's pinout. Bundled with this simulator is a graphic application for creating and testing logical circuits, with Arduino as it's main component. The point of this application is to attach to the running Arduino simulator and visualize the expected Arduino behavior with user's code on a given circuit.

0.1 Comparison of existing solutions

0.2 Specific goals of the thesis

1 Architecture design

1.1 Architecture overview

The whole project is divided into two main parts. The purpose of the backend is to execute the Arduino code and simulate the change on the board's pinout. The purpose of the frontend is to reflect these changes on a user defined logical circuit, while offering the user control over the simulation. We chose to implement the backend in C++ and the frontend in C#. Both backend and frontend are running in its own separate process. There is a technical and an engineering reason for this decision.

The user must be able to have full control over the frontend user interface at all times, even while the Arduino code is being debugged. Every common debugger blocks all running threads of a process, when a breakpoint is hit. If the GUI has been part of the same process as the systems running the simulation, it would freeze and become unresponsive every time a breakpoint is hit. The only way to avoid this behavior is to separate the GUI to a different process. We want to support breakpoints in our project, as they are the most useful tool in a debugger's toolkit.

Splitting the application into multiple processes is actually a common practice among larger pieces of software. The program of each process can be written in a different programming language, they mark clear separation of concerns and force a solid interface design. Each process is easily swappable for another implementation with the same interface while the other processes are still running. In our case, the GUI process could be replaced by a web client, a CLI client, or an IDE add-on.

1.2 Inter-process communication

However, this practice comes with certain challenges. Processes always run concurrently, so a variety of synchronization techniques must be applied to avoid race conditions. The other question is how will the processes exchange data. Process is a kernel level mechanism, thus approaches to handle the exchange can vary between operating systems. These mechanisms are what is called inter-process communication, or IPC for short.

There are more than a few ways to handle IPC on every OS. The ones that we implemented are TCP stream and memory mapping. These are among the most common approaches available both on Windows and Linux. In either case, we must clearly define what the shared data represent. Although memory mapping has more flexibility than TCP, we stuck to sending discrete text or binary messages over time, to keep the implementation simpler and the two approaches interchangable.

In the early iterations, the full state of the Arduino was sent over one message. That included the pinout state, pin modes, timer, etc. The state was periodically requested by the client with a 'Read' message without payload. On a change in the frontend, a 'Write' message with the state as payload would be sent to the server and the values of input pins would be overwritten atomically to the backend's internal state.

This approach was flawed, but easy to implement and synchronize. The main issue was the frequency of reads.

1.2.1 Events

Right now synchronization between processes is maintained by recording every significant event on the Arduino interface, and sending it to the other process with 2 timestamps. Timestamps don't have to be based on a real clock, the only requirement is that it is a non-decreasing sequence of numbers. The first timestamp sent is the simulation time, the time as perceived by the Arduino being simulated. That is time given by a clock that is started right before the setup()

1.2.2 TCP stream

Messages can be sent and received through a stream of data on a TCP connection. TCP is commonly used to send data over a network, meaning backend and frontend processes can run on different machines. But listening to a connection on the same machine works just as well.

The backend implements the TCP server, while the frontend implements the client, both connected to the same port. TCP messages are guaranteed to arrive in the same order as they were sent.

1.2.3 Memory mapping

1.3 Backend

The choice for backend was obvious, the Arduino code is a syntactically valid C++ code.

1.4 Frontend

2 Implementation

2.1 Backend

2.1.1 Simulator module

2.1.2 Executable module

2.1.3 IPC Adapters

TPC Adapter module

Shared Memory Adapter module

2.2 Frontend

2.2.1 Component Management project

2.2.2 Main project with Avalonia UI

2.2.3 IPC Adapter project

3 User documentation

3.1 Compiling and running Arduino code

3.2 Defining components

3.3 Defining scenes

Conclusion

List of Figures

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment